



CLUSTRO

Social Network — DSA Implementation C++ Social Media Simulation

Team Members:

Maliha Shah (09-134242-020)

Laiba Rashid (09-134242-018)

Deena Fatima (09-134242-011)

Iftikhar Ahmed (09-134242-016)

Clustro – Mini Social Network

Platform

Console-based C++ application

Storage

File-based persistence (text files)

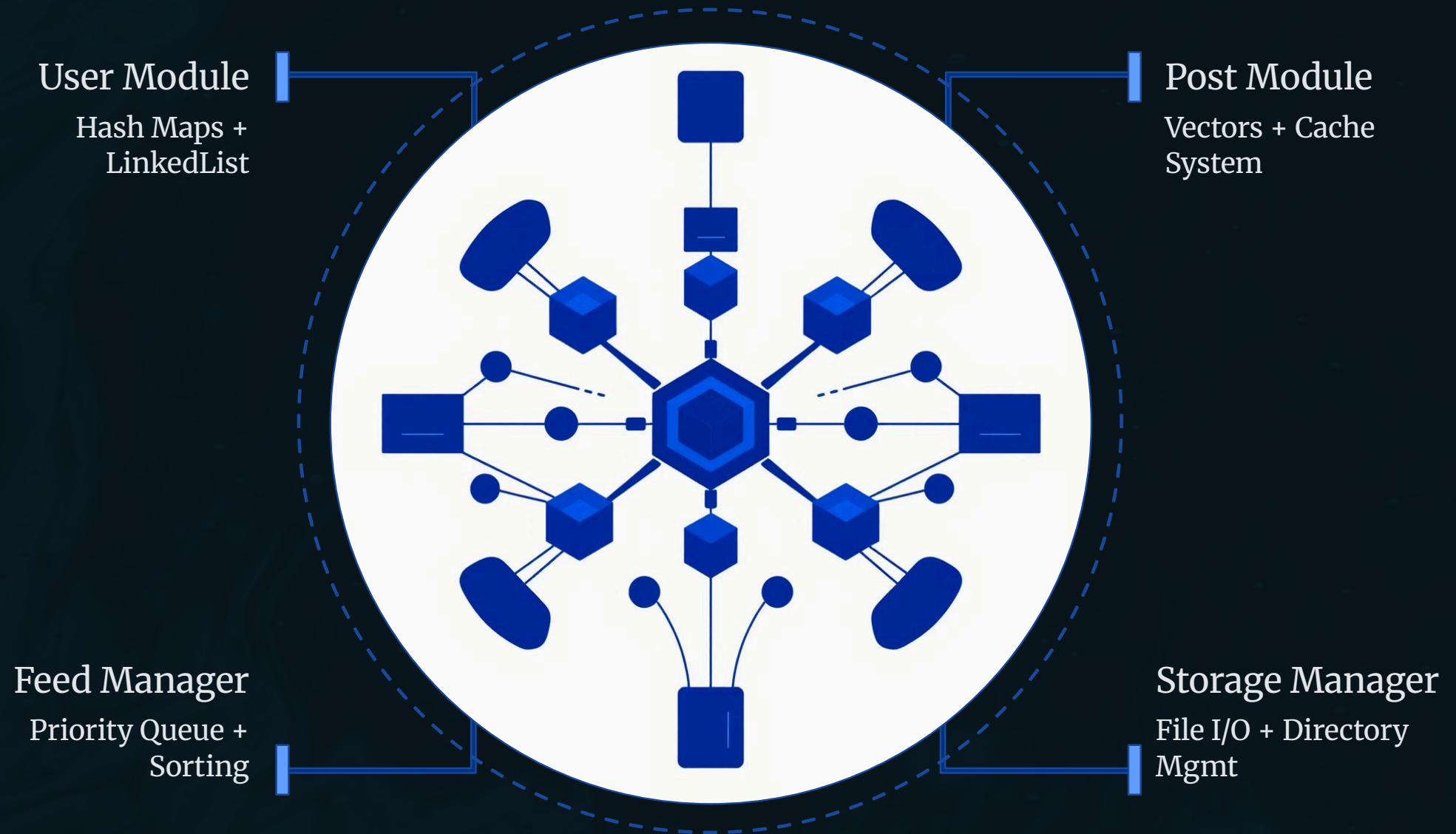
Architecture

Modular OOP with Manager classes

Purpose

Educational demonstration of DSA in social networking context

System Architecture



Clustro is a fully-featured offline social networking platform built entirely in C++ using various Data Structures and Algorithms. The project simulates core social media functionalities with persistent file storage.

📁 File-by-File Detailed Analysis

1. user.hpp – User Management System

Purpose: Handles user authentication, profiles, and user data management.

Key Features:

User Class:

- Unique ID generation using `clustro::generateID(8)`
- User attributes: ID, username, email, password, bio, post count, followers/following counts
- Serialization/deserialization with `stringify()` and `parse()` methods
- Signup functionality with password confirmation
- Getters/setters for all attributes

UserManager Class:

- [DSA Used: Hash Maps \(unordered_map\), LinkedList](#)
- Triple-index system for O(1) lookups:
 - `users_by_id`: Map user ID → User*
 - `users_by_email`: Map email → User*
 - `users_by_username`: Map username → User*
- Index Management: Automatically builds indexes from `LinkedList`
- Search functions with both verbose and silent versions
- Thread-safe index management with `index_dirty` flag

Technical Highlights:

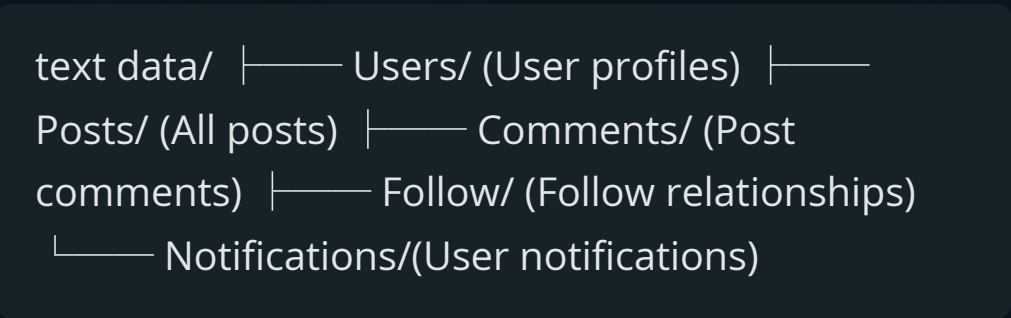
```
cpp // Triple hash map indexing for fast lookups inline static std::unordered_map users_by_id; inline static
std::unordered_map users_by_email; inline static std::unordered_map users_by_username;
```

2. storage.hpp – File System Management

Purpose: Handles all file I/O operations for persistent storage.

Key Features:

Directory Structure:



Storage Class:

- Generic `save_item()` and `load_items()` template methods
- Creates necessary directories on initialization
- Individual save methods for each data type
- Safe file operations with error handling
- Notification block management

Technical Highlights:

```
cpp // Generic loading function using LinkedList template clustro::LinkedList load_items(const std::string
&subdir)
```

3. post.hpp – Post Management System

Purpose: Handles social media posts (creation, editing, deletion).

Key Features:

Post Class:

- Attributes: user_id, content, ID, timestamp, likes, comments count
- Auto-generated ID and timestamp on creation
- Serialization with stringify() method
- Like/comment increment/decrement methods

PostManager Class:

- DSA Used: Vector (for caching), File I/O
- Post creation with automatic saving
- In-memory cache (posts_cache) for faster access
- Post editing and deletion with ownership verification
- Like/unlike functionality

Technical Highlights:

```
cpp std::vector posts_cache; // Cache for faster access void refresh_cache(); // Updates cache from storage
```

4. comment.hpp – Comment Management System

Purpose: Handles comments on posts with moderation features.

Key Features:

Comment Class:

- Attributes: post_id, user_id, content, likes, timestamp
- Moderation Features: Edited flag, deleted flag, report system
- Report tracking with reporter IDs and reasons
- Timestamp parsing and formatting utilities
- Age checking with is_older_than() method

CommentManager Class:

- Comment creation with post association
- Loading comments by post or user
- Like/unlike functionality
- Comment editing with ownership verification
- Report system for inappropriate content

Technical Highlights:

```
cpp // Moderation features bool edited; bool deleted_flag; int report_count; std::vector reports;
```

5. follow.hpp & followmanager.hpp – Social Graph System

Purpose: Manages follow/unfollow relationships using graph concepts.

Key Features:

Follow Class:

- Simple relationship storage: follower_id → followee_id
- Unique ID generation: followee_id + "->" + follower_id
- Timestamp tracking

FollowManager Class:

- DSA Used: Graph (Adjacency List), LinkedList, Hash Maps
- FollowNode structure with following/followers LinkedLists
- Graph representation: unordered_map
- Smart Suggestions Algorithm:
 - 2nd-degree connections (friends of friends)
 - Scoring system based on mutual connections
 - Random user suggestions when needed
- No self-following prevention

Technical Highlights:

```
cpp // Graph representation struct FollowNode { std::string user_id; clustro::LinkedList<std::string> following; // Outgoing edges clustro::LinkedList<std::string> followers; // Incoming edges };
```

6. notification.hpp – Notification System

Purpose: Handles user notifications for various activities.

Key Features:

Notification Class:

- Enum-based notification types: LIKE, COMMENT, FOLLOW, MESSAGE, SYSTEM
- Attributes: sender_id, user_id, message, type, read status
- Mark as read functionality
- Serialization with stringify()

NotificationController Class:

- Load/save notifications using Storage
- Get unread notifications count
- Mark all as read functionality
- Notification deletion

Technical Highlights:

```
cpp enum class NotificationType { LIKE, COMMENT, FOLLOW, MESSAGE, SYSTEM, UNKNOWN };
```

7. feedmanager.hpp – Personalized Feed System

Purpose: Generates personalized feeds for users.

Key Features:

FeedManager Class:

- DSA Used: Priority Queue (conceptually), Sorting
- Generates feed from followed users' posts
- Chronological sorting (newest first)
- Integrates UserManager, FollowManager, and PostManager

Algorithm:

- Get list of followed users
- Filter posts by followed users
- Sort by timestamp (descending)
- Return personalized feed

8. clustro.hpp & clustro.cpp – Main Application

Purpose: Main application controller and user interface.

Key Features:

Clustro Class:

- Initializes all managers
- Main event loop
- Console-based user interface
- Session management with `current_user`

UI Features:

- Login/Signup system
- Main menu with options:
 - Create Post
 - View Feed
 - Follow/Unfollow
 - View Profile
 - Logout
- Color-coded console output
- Screen clearing and pause functionality

Technical Highlights:

```
cpp // Main application flow void run() { while (true) { showLoginMenu(); if (current_user == nullptr) break; showMainMenu(); } }
```

9. utils/ Directory – Utility Functions

Purpose: Contains helper utilities used throughout the project.

Key Features (assumed based on usage):

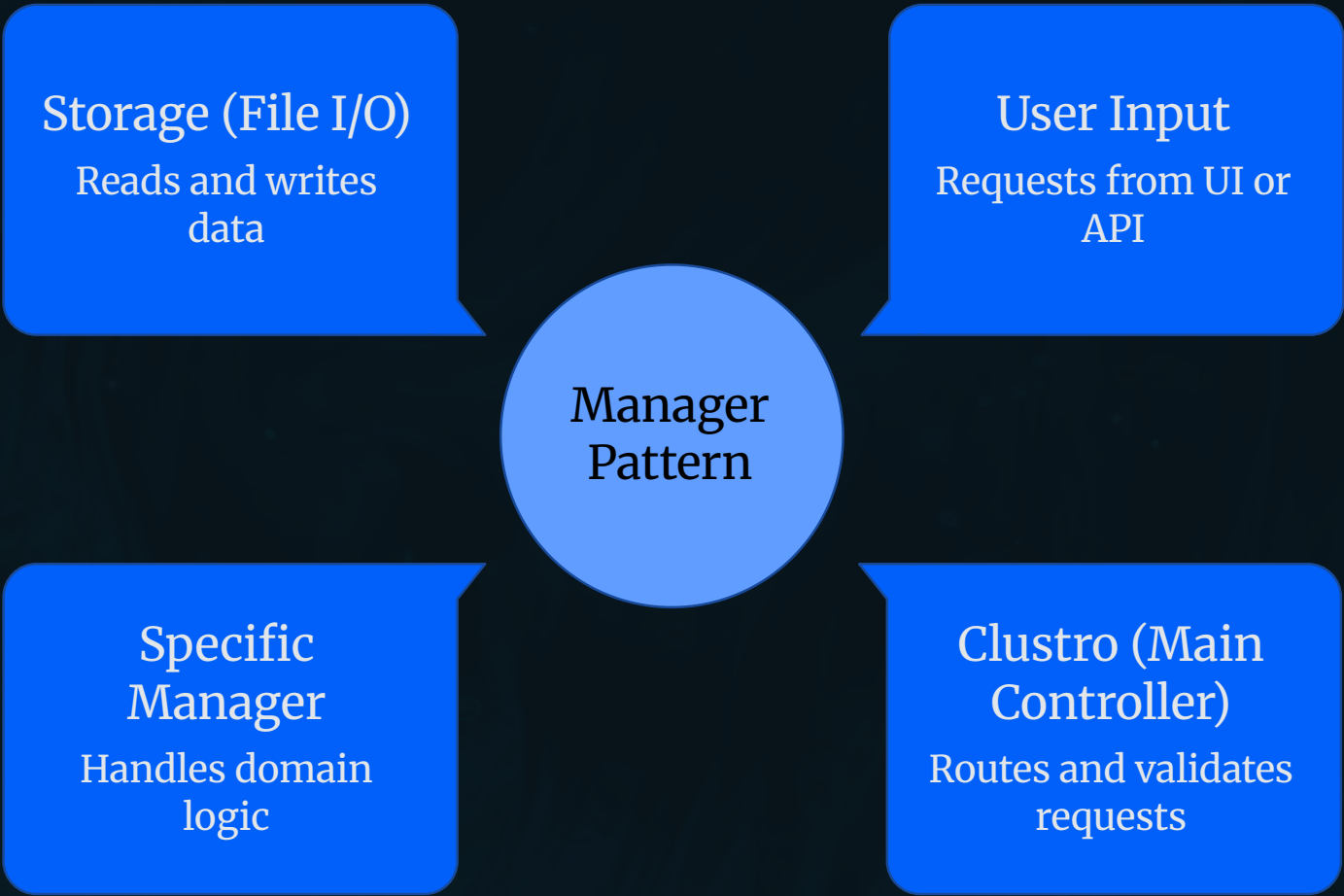
- `utils.hpp`: Color codes, ID generation, timestamp utilities
- `LinkedList.hpp`: Custom `LinkedList` implementation
- Other utility functions for string manipulation, validation, etc.

DSA Concepts Used in Project

Hash Maps	UserManager, FollowManager	O(1) user lookups, graph representation
Linked Lists	Follow relationships, user lists	Store variable-length sequences
Vectors	Post/comment caching	Dynamic arrays for fast access
Graph	FollowManager (Adjacency List)	Represent social connections
Priority Queue	FeedManager (conceptual)	Sort posts by timestamp
Sorting Algorithms	Feed generation, suggestions	Order posts, rank suggestions



Data Flow Architecture



Manager Pattern:

- Each entity (User, Post, Comment, etc.) has its own Manager class
- Managers handle business logic and delegate storage to Storage class
- Separation of concerns for clean architecture



Security & Validation Features

Password Confirmation

during signup

Unique Constraints

Email and username
uniqueness

Ownership Verification

Users can only edit/delete their
own content

No Self-following

prevention

Input Validation

throughout the system



Storage System Design

File Format:

```
text id:ABC123 username:johndoe email:john@email.com password:mypassword123 about:Software  
Developer followers:10 following:5 posts:3
```

Advantages:

- Human-readable files
- Easy debugging
- Simple parsing with : delimiter
- No database dependency



Key Algorithms Implemented

- **User Search:** $O(1)$ using hash maps
- **Feed Generation:** $O(n \log n)$ sorting
- **Follow Suggestions:** $O(n^2)$ for mutual connection scoring
- **Graph Operations:** $O(1)$ for follow/unfollow
- **Caching:** Vector-based caching for performance

Project Strengths



Modular Design

Each component is independent



Scalable Architecture

Easy to add new features



Educational Value

Demonstrates real-world DSA applications



No External Dependencies

Pure C++ with STL



Persistent Storage

All data survives program restart



Error Handling

Comprehensive try-catch blocks



Clean Code

Well-documented with clear separation of concerns

Potential Improvements

- **Password Hashing:** Currently stores plaintext passwords
- **Database Integration:** Could use SQLite for complex queries
- **Multithreading:** For concurrent user access
- **GUI Interface:** Replace console with graphical UI
- **Network Support:** Add client-server architecture
- **Advanced Algorithms:** Implement more sophisticated feed ranking

Learning Outcomes

This project demonstrates:

- Real-world application of DSA concepts
- File-based persistence systems
- Object-oriented design patterns
- Modular software architecture
- User interface design (console-based)
- Error handling and input validation
- System design thinking